

建立MVC專案

•	3-1: 開發環境	
	安裝ASP.NET MVC開發環境	3-2
•	3-2: 建立MVC專案	
	建立ASP.NET MVC專案(Framework)	3-4
	建立ASP.NET Core MVC專案	3-7
	執行Web應用程式	3-10
	ASP.NET 網址比較	3-11
	.NET Core MVC專案基本架構	3-12
	Program.cs – 依賴注入 (DI)	3-14
	Program.cs –中間件Middleware	3-15
	網址規則Route	3-16
	Attribute Routing	3-17
	加入Controller	3-18
	加入Action	3-20
	加入View	3-21

Module 03

安裝ASP.NET MVC開發環境 -1



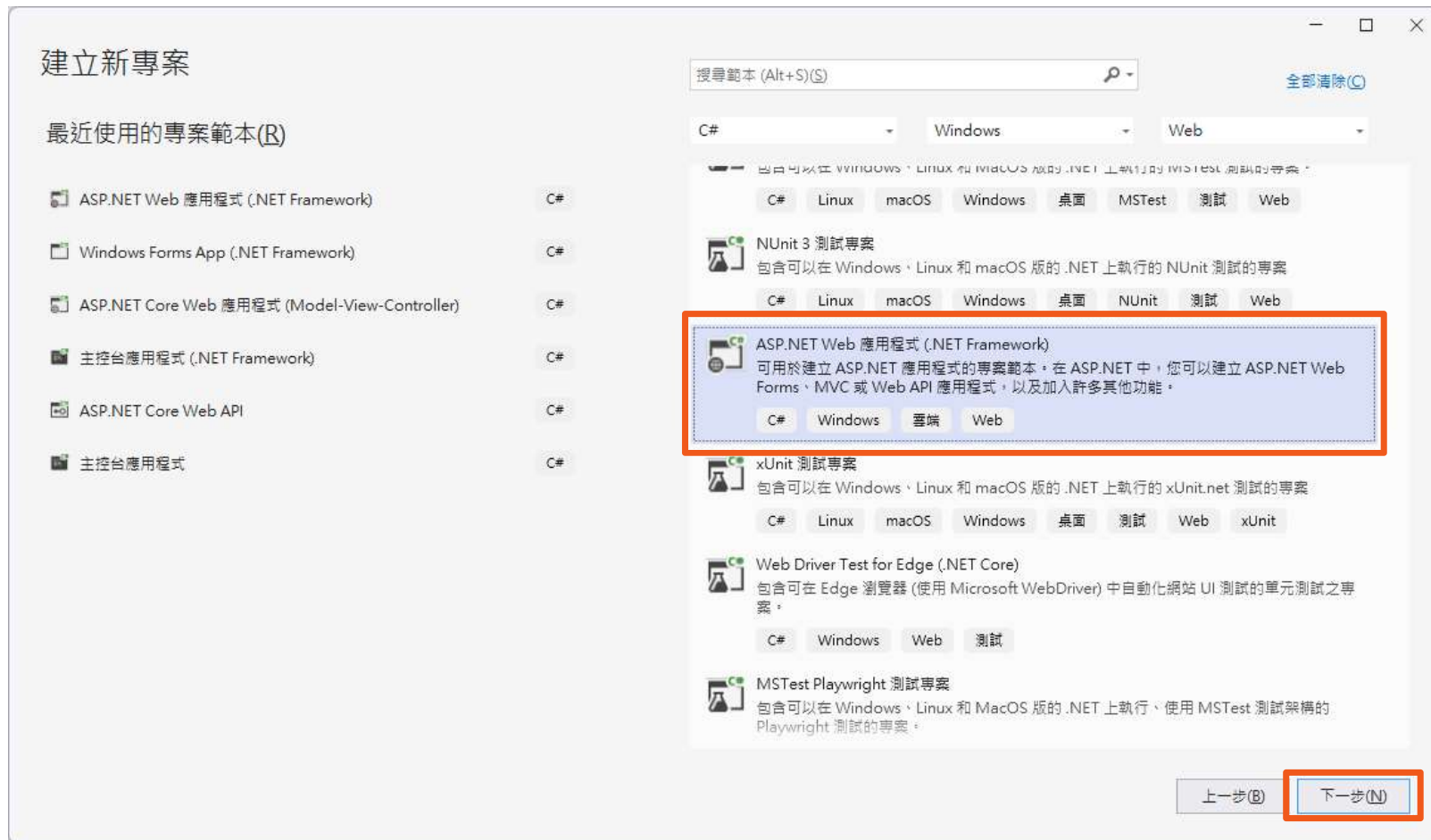
- 開啟Visual Studio Installer，按下[修改]。

安裝ASP.NET MVC開發環境 -2



- 勾選[ASP.NET與網頁程式開發]，並按[在下載時安裝]。

建立ASP.NET MVC專案(Framework) -1



- 選取[ASP.NET Web應用程式(.NET Framework)]範本，並按[下一步]。

建立ASP.NET MVC專案(Framework) -2

設定新的專案

ASP.NET Web 應用程式 (.NET Framework) C# Windows 雲端 Web

專案名稱(N)

位置(L) ...

解決方案名稱(M) ⓘ

☐ 將解決方案與專案置於相同目錄中(D)

架構(F)

專案 將在「D:\工作\台南各項-OneDrive\OneDrive - ispan.com.tw\工作\備課\ASP.NET Core MVC\FrameworkMVC\FrameworkMVC\FrameworkMVC\」中建立

上一步(B) **建立(C)**

- 設定各項欄位，並按[建立]。

建立ASP.NET MVC專案(Framework) -3

建立新的 ASP.NET Web 應用程式

- 空白**
用來建立 ASP.NET 應用程式的空白專案範本。範本中不含任何內容。
- Web Form**
用來建立 ASP.NET Web Forms 應用程式的專案範本。ASP.NET Web Forms 可讓您使用熟悉的拖放、事件驅動模型來建置動態網站。設計介面及數以百計的控制項和元件可讓您迅速建置精緻、功能強大且具備資料存取權限的 UI 驅動網站。
- MVC**
用於建立 ASP.NET MVC 應用程式的專案範本。ASP.NET MVC 可讓您使用「模型-檢視-控制器」架構來建置應用程式。ASP.NET MVC 包括多個功能，可啟用快速且測試驅動的開發，來建立使用最新標準的應用程式。
- Web API**
用於建立 RESTful HTTP 服務的專案範本，而服務可以連線較大範圍的用戶端 (包括瀏覽器和行動裝置)。
- 單一頁面應用程式**
使用 ASP.NET Web API 來建立豐富用戶端 JavaScript 驅動的 HTML5 應用程式的專案範本。單一頁面應用程式提供豐富的使用者經驗，包括使用 HTML5、CSS3 和 JavaScript 的客戶端互動。

驗證

無

個人帳戶
Microsoft 身分識別平台
Windows

新增資料夾和核心參考

- ☐ Web Form(F)
- ☒ MVC(M)
- ☐ Web API(W)

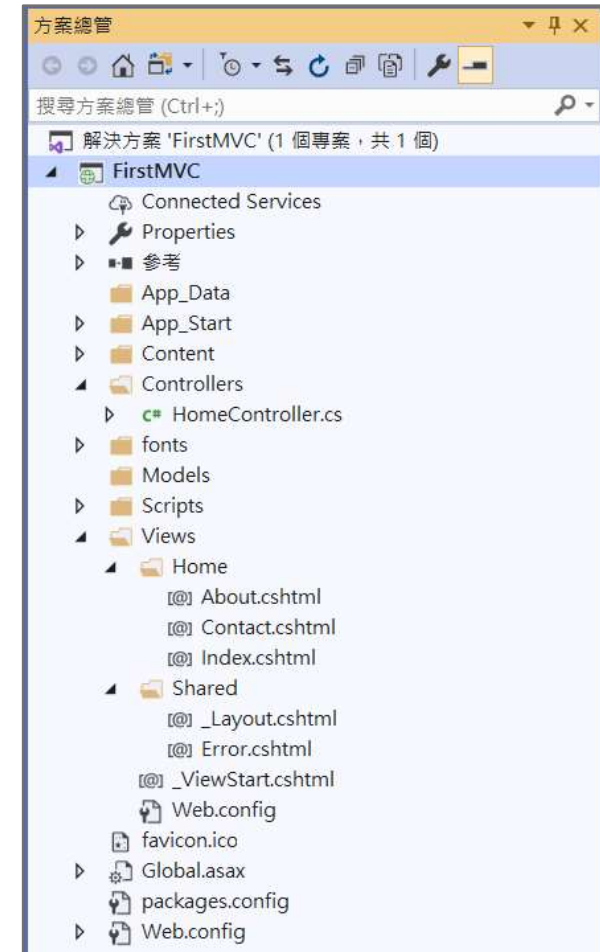
進階

- ☒ 設定 HTTPS(C)
- ☐ 容器支援
(需要 [Docker Desktop](#))
- ☐ 也建立單元測試的專案(U)

FrameworkMVC.Tests

上一步(B)

建立

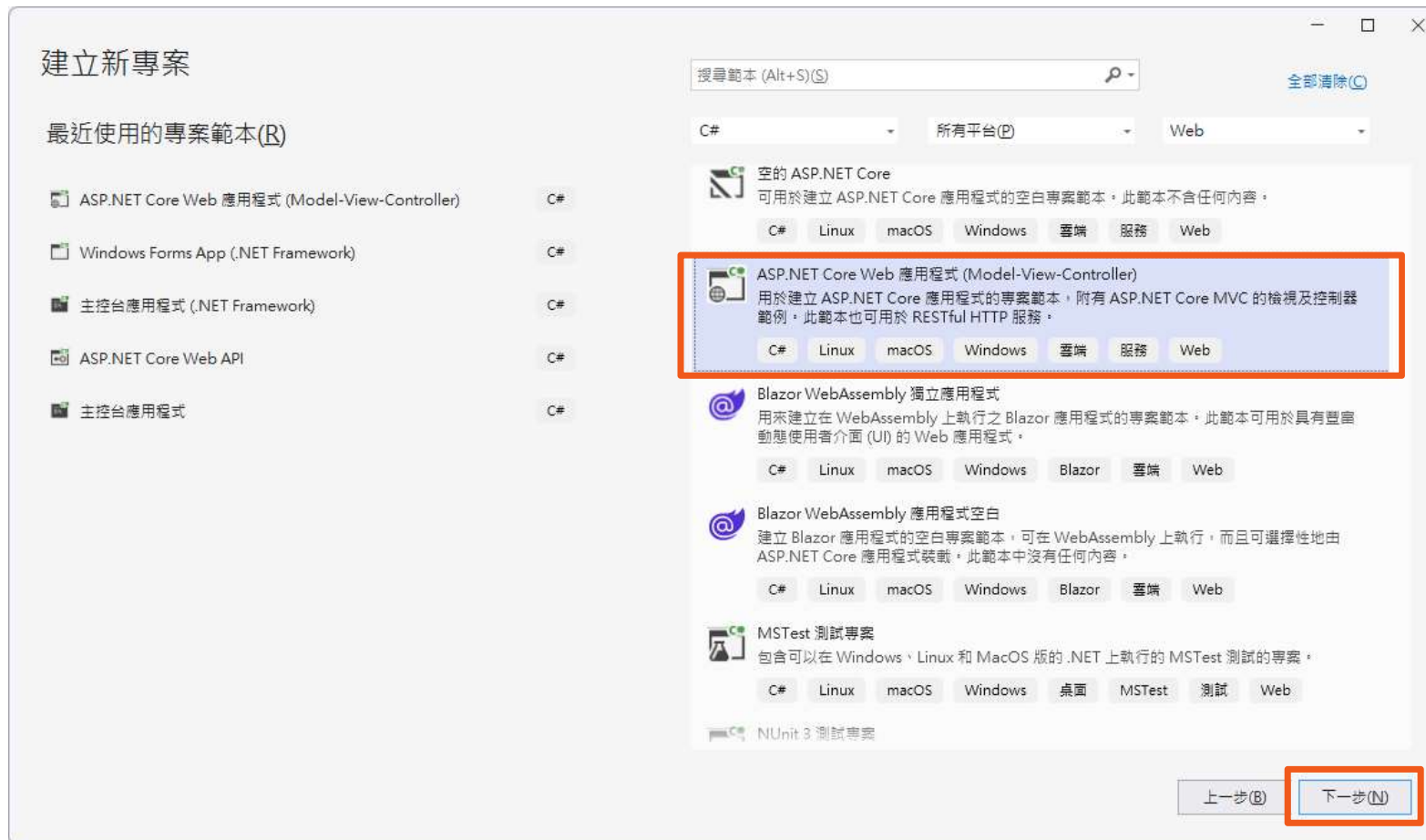


- 設定各項欄位，並按[建立]。

3-6

張佑晨 編

建立ASP.NET Core MVC專案 -1



- 選取[ASP.NET Core Web應用程式範本，並按[下一步]。

建立ASP.NET Core MVC專案 -2

設定新的專案

ASP.NET Core Web 應用程式 (Model-View-Controller) C# Linux macOS Windows 雲端 服務 Web

專案名稱(P)

FirstCoreMVC

位置(L)

D:\工作\台南各項-OneDrive\OneDrive - ispan.com.tw\工作\備課\ASP.NET Core MVC

解決方案名稱(M) ⓘ

FirstCoreMVC

☐ 將解決方案與專案置於相同目錄中(D)

專案 將在「D:\工作\台南各項-OneDrive\OneDrive - ispan.com.tw\工作\備課\ASP.NET Core MVC\FirstCoreMVC\FirstCoreMVC\」中建立

上一步(B) 下一步(N)

- 設定各項欄位，並按[下一步]。

建立ASP.NET Core MVC專案 -3

其他資訊

ASP.NET Core Web 應用程式 (Model-View-Controller) C# Linux macOS Windows 雲端 服務 Web

架構(E) ⓘ
.NET 10.0 (長期支援)

驗證類型(A) ⓘ
無
無
個人帳戶
Microsoft 身分識別平台
Windows

☒ 針對 HTTPS 進行設定(H) ⓘ

☐ 啟用容器支援(E) ⓘ

容器作業系統(O) ⓘ
Linux

容器組建類型 ⓘ
Dockerfile

☐ 不要使用最上層陳述式(I) ⓘ

☐ 在 Aspire 協調流程中登錄 ⓘ

Aspire 版本 ⓘ
13.4

2026年8月 功能更新

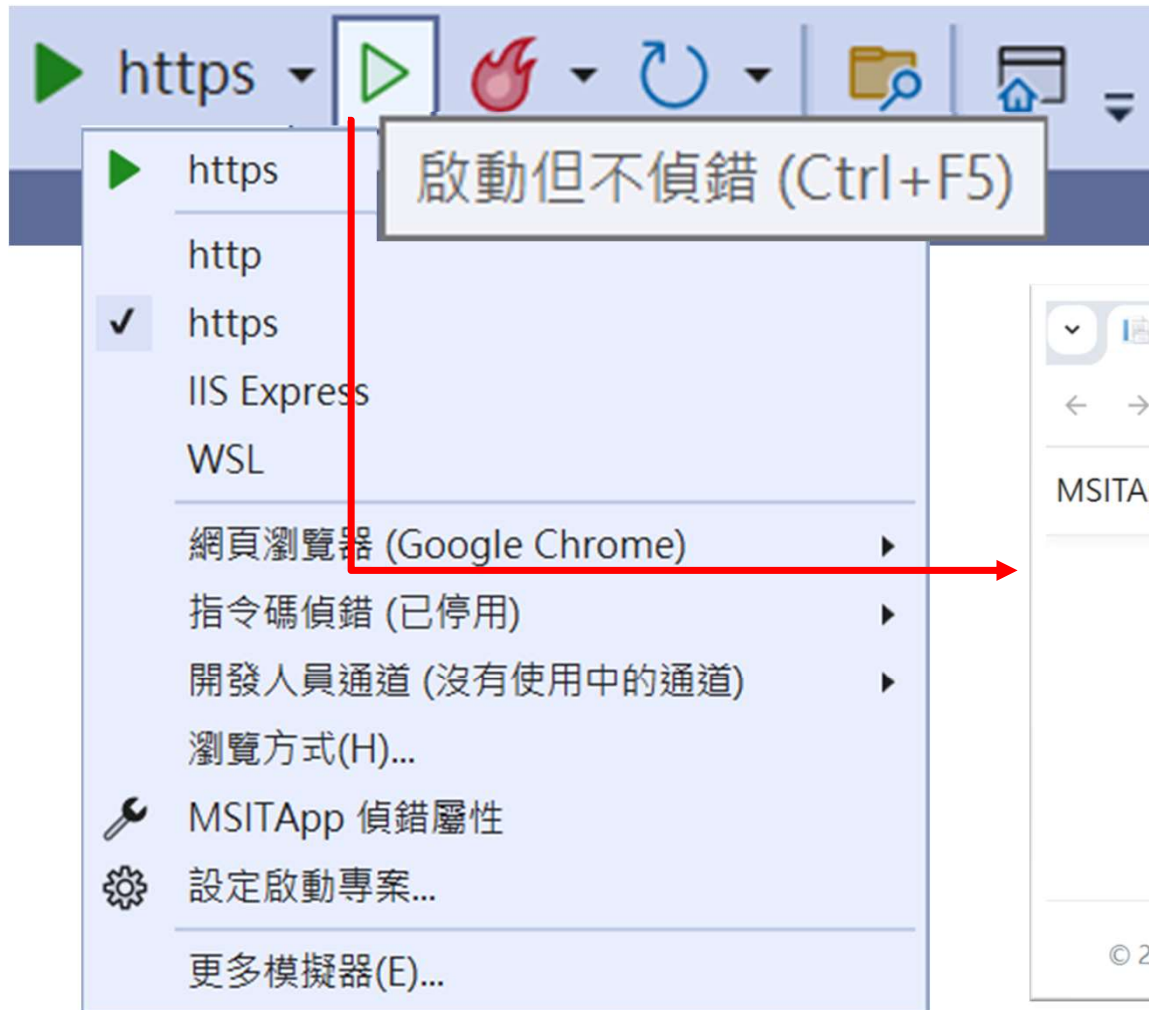
上一步(B) 建立(C)

- 設定各項欄位，並按[建立]。

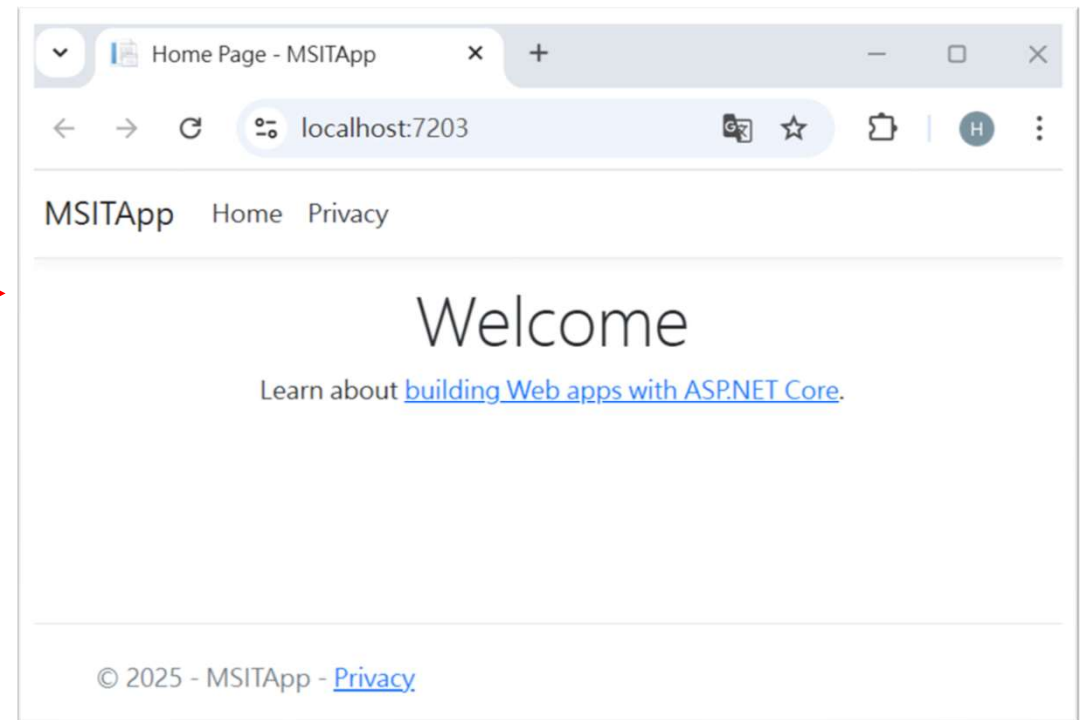
3-9

張佑晨 編

執行Web應用程式



- 偵錯(F5)
- 啟動但不偵錯(Ctrl+F5)



ASP.NET 網址比較

- ASP.NET 網址

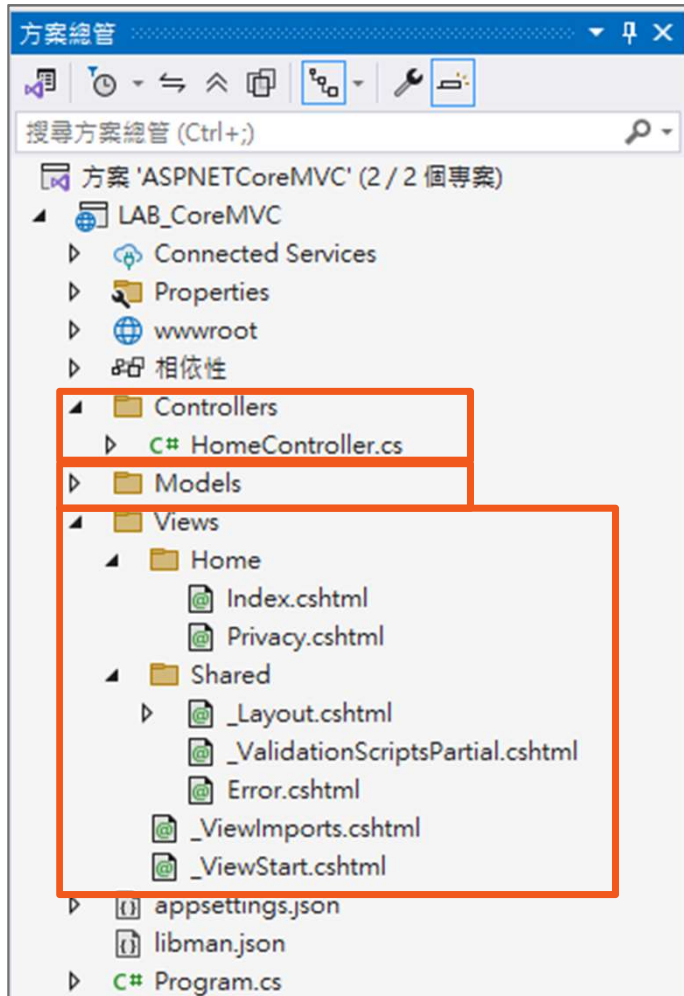
```
http://localhost/site/accunt/login.aspx
```

- ASP.NET MVC 網址

```
http://localhost/account/login
```

```
http://伺服器位置(或Domain Name)/controller名稱/action名稱/參數
```

.NET Core MVC專案基本架構-1



- **Controllers**

- 存放控制器（Controller），負責接收 HTTP 請求並回傳 View 或資料。

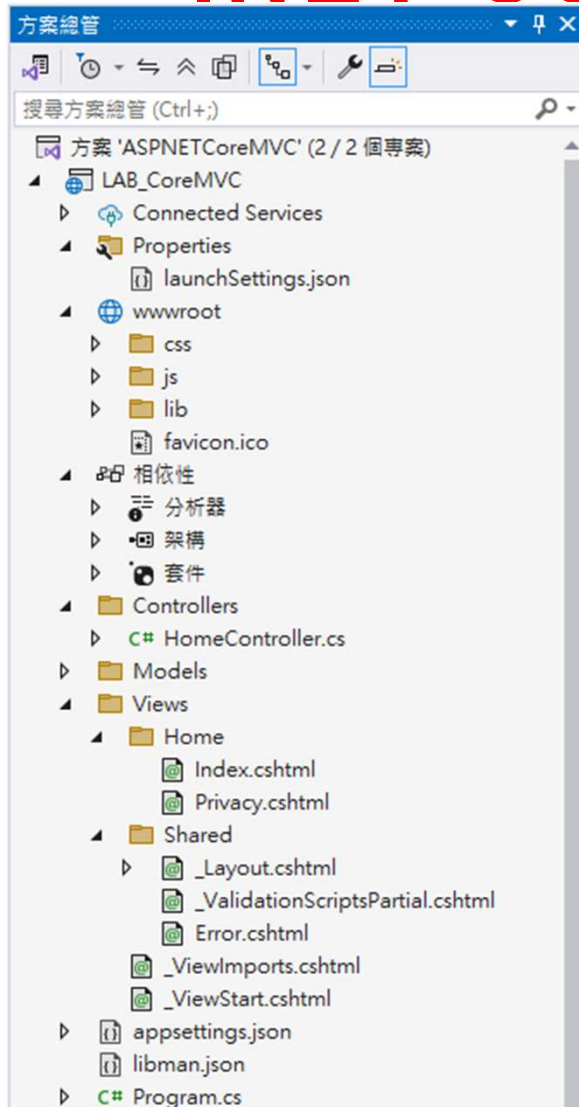
- **Models**

- 存放資料模型與商業邏輯相關類別（Model），可包含 ViewModel、Entity 等。

- **Views**

- 存放畫面檔案 Razor：.cshtml，用於呈現使用者介面。
- Shared存放共用畫面，例如：
 - 主版頁面（Layout：_Layout.cshtml）
 - 共用元件（Partial View：_Partial.cshtml）

.NET Core MVC專案基本架構-2



- **launchSettings.json**
 - 本機開發環境的啟動設定檔，用於設定執行網址、環境變數 (Environment) 等資訊。
- **wwwroot**
 - 靜態檔案資料夾，用來存放 CSS、JavaScript、圖片、字型 等前端資源。
- **相依性 (Dependencies)**
 - 管理專案所使用的伺服器端程式庫，主要透過 NuGet 套件 進行管理。
- **appsettings.json**
 - 應用程式的組態設定檔，常用於設定連線字串、系統參數與環境設定。
- **Program.cs**
 - 應用程式的進入點，負責設定DI Container與Middleware 管線，並啟動 Web 應用程式。

Program.cs – 依賴注入 (DI)

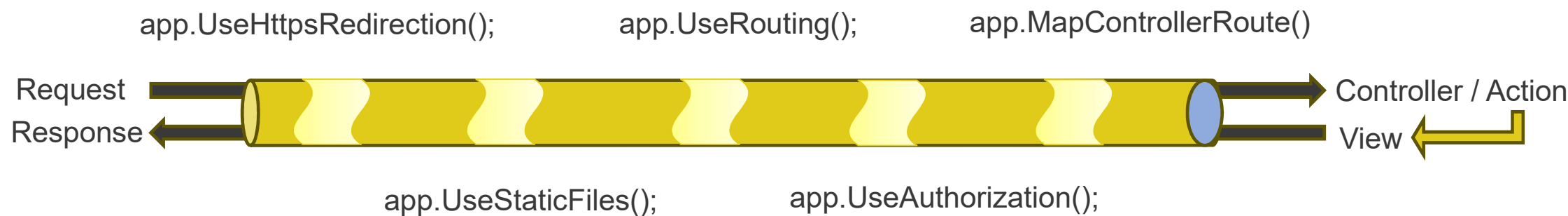
- 在 ASP.NET Core MVC 中，Program.cs 會負責建立應用程式並設定服務容器。通常會在這裡把應用程式所需的服務與物件註冊到 DI Container，讓系統在執行期間能：
 - 自動解析相依性 (Controller / Service 需要什麼就自動注入)
 - 集中管理生命週期 (Singleton / Scoped / Transient)
 - 降低耦合、提升可測試性 (方便替換實作、Mock、單元測試)

常見註冊：

```
builder.Services.AddControllersWithViews();  
builder.Services.AddDbContext<...>();  
builder.Services.AddScoped<IService, Service>();
```

Program.cs – 中間件 Middleware

- ASP.NET Core 中，每一個 Middleware (中間件) 都代表請求 (Request) 與回應 (Response) 流程中的一個處理階段。
- 常見的處理階段包含：路由 (Routing)、靜態檔案 (Static Files)、身分驗證/授權 (AuthN/AuthZ)、Session (UseSession)、CORS (UseCors) 等。
- Middleware 是「管線」，每一站負責一種功能 (路由、靜態檔、權限、Session、CORS...)。
- 順序決定行為：先註冊先處理 Request，回應時反向回傳。



網址規則Route

- 網址URL路徑對應關係是透過路由Route來定義的。
- 預設路由在Program.cs檔案中。

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

- 自訂路由：

```
app.MapControllerRoute(  
    name: "products",  
    pattern: "products/{action=Index}/{id?}",  
    defaults: new { controller = "Test" });
```

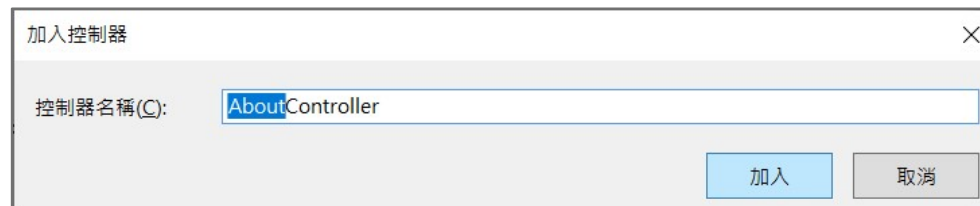
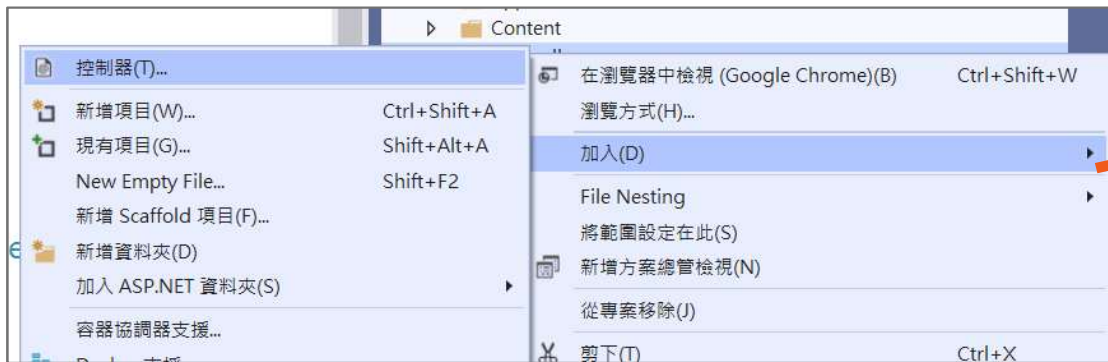
Attribute Routing

- 常用於 Web API 開發中，透過在 Controller / Action 上套用 [Route] 屬性，明確定義 哪些 URL 可以對應到該 Controller 或 Action。
- 同時也可以在路由屬性中：
 - 定義 路由參數設定
 - 參數限制 (Constraints)
 - 控制 HTTP 方法 (GET / POST / PUT / DELETE)

```
[Route("Default")]  
[Route("Index")]  
[Route("")]  
public IActionResult Index()  
{  
    return View()  
}
```

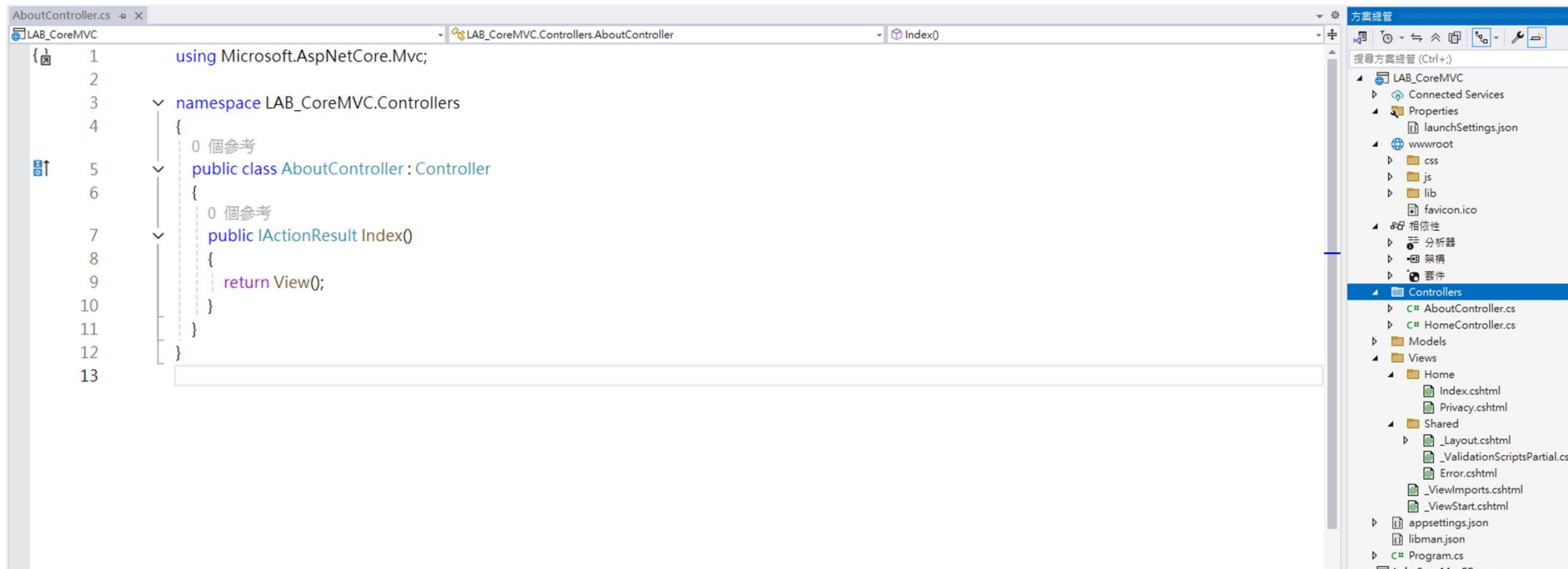
加入 Controller -1

- Controller用於控制網站的流程，決定該使用的Model與回傳的View。
- 於Controllers資料夾按下右鍵 > 加入 > 控制器 > 選擇指定樣版 > 輸入控制器名稱 > [加入]



加入Controller -2

- 加入控制器後將會得到一個cs檔，檔案中的類別就是控制器類別。



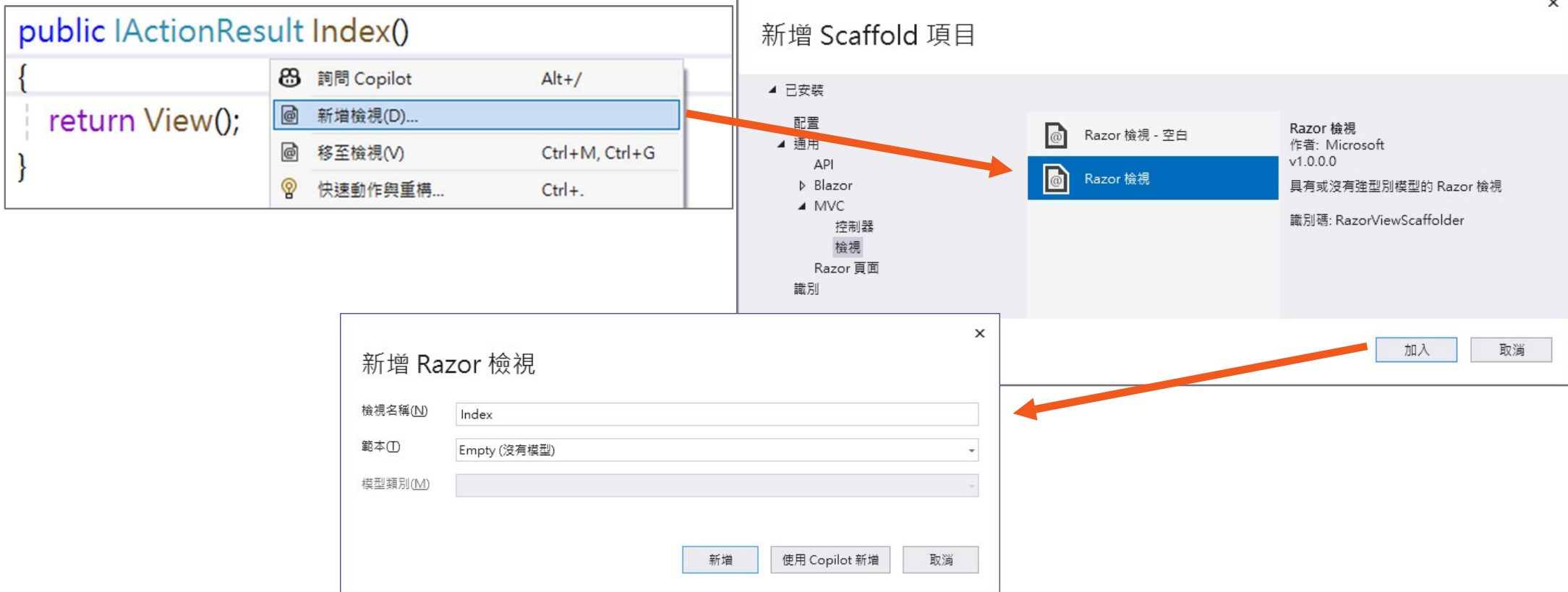
加入Action

- 控制器類別下的 IActionResult 方法即為Action。

```
namespace LAB_CoreMVC.Controllers
{
    0 個參考
    public class AboutController : Controller
    {
        0 個參考
        public IActionResult Index()
        {
            return View();
        }
    }
}
```

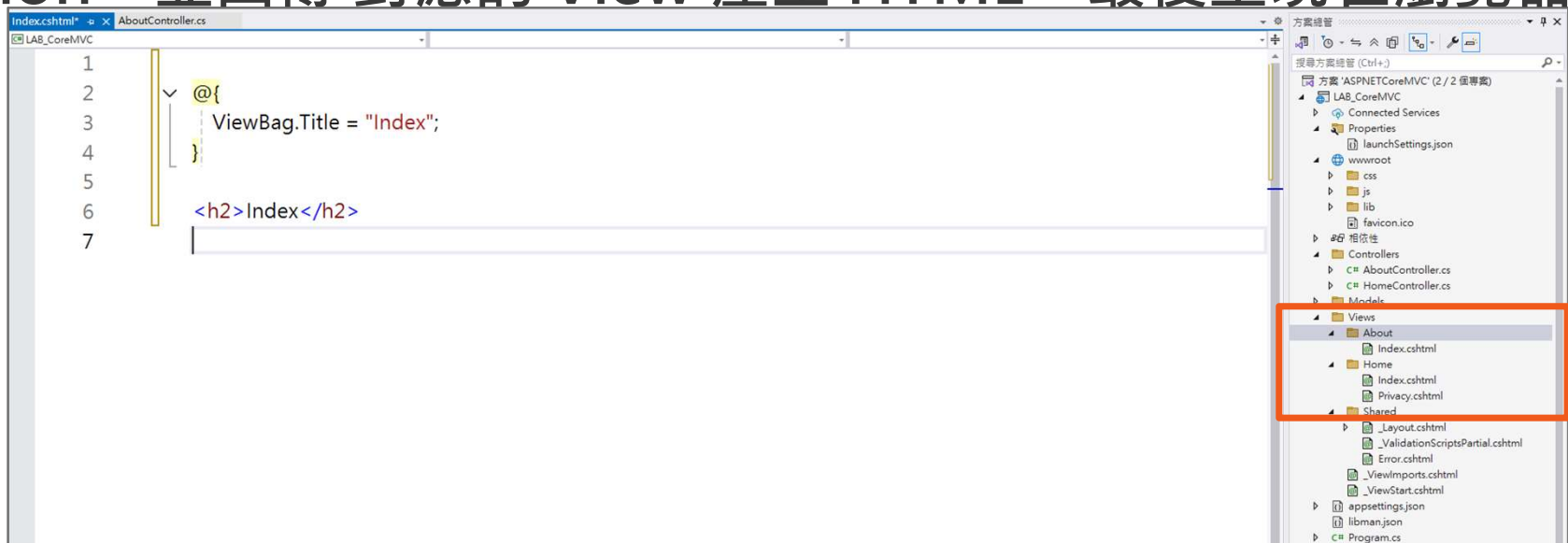
加入 View -1

- 於指定的Action上按下右鍵 > 新增檢視 > 選擇樣版 > 各式設定 > 加入。



加入 View -2

- 新增 View 後，會在 Views 資料夾下看到對應新增的 .cshtml 檔案。 .cshtml 是 ASP.NET 的 Razor 檢視檔，負責呈現使用者介面（UI）與顯示資料。
- 當使用者透過網址呼叫某個 Controller/Action 時，伺服器會執行該 Action，並回傳對應的 View 產生 HTML，最後呈現在瀏覽器上。



加入 View -3

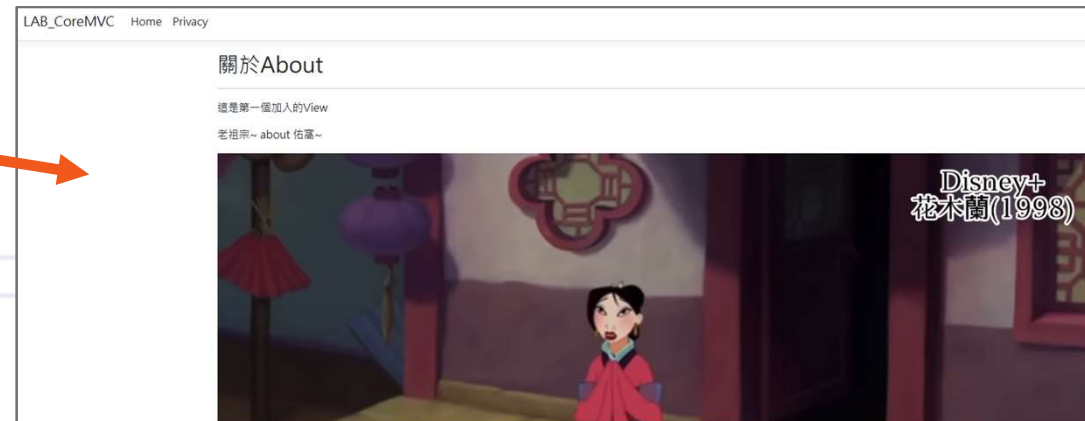
- 嘗試在View加入HTML、CSS與JavaScript。

```
Controller.cs  Index.cshtml  AboutController.cs
@{
    ViewBag.Title = "Index";
}

<h2>關於About</h2>
<hr />
<p>這是第一個加入的View</p>

<p>老祖宗~ about 佑窩~</p>

```



加入 View -4

- Razor 檢視檔 (.cshtml) 會使用 @ 作為語法標記，用來在同一份檔案中嵌入 C# 程式碼，並與 HTML 標籤做區分與整合。
- 常見用途包含：輸出變數、條件判斷、迴圈與呼叫 Razor Helper 等。

```
@{  
    ViewBag.Title = "Index";  
}  
  
<h2>關於 About </h2>  
  
<hr />  
<p>這是使用ASP.NET MVC加入的第一個View</p>  
<p>IU真的好可愛!!!</p>  
  
  
  
<p>現在時間：@DateTime.Now.ToString()</p>
```